

Experiment No. 1

Encryption and Decryption Program in C

Program in C:

```
#include <stdio.h>
int main()
{
    int i, x;
    char str[100];

    printf("\nPlease enter a string:\t");
    gets(str);

    printf("\nPlease choose following options:\n");
    printf("1 = Encrypt the string.\n");
    printf("2 = Decrypt the string.\n");
    scanf("%d", &x);

    //using switch case statements
    switch(x)
    {
    case 1:
        for(i = 0; (i < 100 && str[i] != '\0'); i++)
            str[i] = str[i] + 3; //the key for encryption is 3 that is added to ASCII value

        printf("\nEncrypted string: %s\n", str);
        break;

    case 2:
        for(i = 0; (i < 100 && str[i] != '\0'); i++)
            str[i] = str[i] - 3; //the key for encryption is 3 that is subtracted to ASCII value

        printf("\nDecrypted string: %s\n", str);
        break;

    default:
        printf("\nError\n");
    }
    return 0;
}
```

Output:

Please enter a string: **hello**
Please choose following options:
1 = Encrypt the string.
2 = Decrypt the string.
1

Encrypted string: **khoor**

Please enter a string: **khoor**
Please choose following options:
1 = Encrypt the string.
2 = Decrypt the string.
2

Decrypted string: **hello**

Experiment No. 2(a)

Enter Email ID & Password

Program in C++:

```
#include <iostream>

using namespace std;

class LoginManager{
public:
    string userNameAttempt;
    string passWordAttempt;

    LoginManager(){
        accessGranted = 0;
    }

    void login(){
        cout << "Hey you need to enter your password and user.\nUsername:";
        cin >> userNameAttempt;
        if(userNameAttempt==userName){
            cout << "Password:";
            cin >> passWordAttempt;

            if(passWordAttempt==passWord){
                cout << "Hey, that's right.";
            }
        }
    }
private:
    string passWord = "easypassword";
    string userName = "user@email.com";
    bool accessGranted;
};

int main()
{
    LoginManager loginManagerObj;
    loginManagerObj.login();
}
```

Output:

```
Hey you need to enter your password and user.
Username:user@email.com
Password:easypassword
Hey, that's right.
```

Experiment No. 2(b)

Enter Username & Password

Program in C:

```
#include <stdio.h>
#include <string.h>
#include <conio.h>

//created by chaitanya

int main()
{
    char username[15];
    char password[12];

    printf("Enter your username:\n");
    scanf("%s",&username);

    printf("Enter your password:\n");
    scanf("%s",&password);

    if(strcmp(username,"manishdabhade")==0){
        if(strcmp(password,"admin")==0){

            printf("\nWelcome.Login Success!");

        }else{
            printf("\nwrong password");
        }
    }else{
        printf("\nUser doesn't exist");
    }

    return 0;
}
```

Output:

```
Enter your username:
greatapple598
Enter your password:
admin
```

```
Welcome.Login Success!
```

Experiment No. 2(c)

Hiding Password

Program in C++:

```
#include <iostream>
#include <string>

using namespace std;
void askForSecretPassword(void){
    string password;
    cout<<"Enter your password: ";
    cin>>password;
    int len=password.length();
    system("cls");
    cout<<"Enter numeric password: ";
    for(int i=0;i<len;i++){
        cout<<"*";
    }cout<<endl;
};
int main(){

    askForSecretPassword();
    system("pause");
    return 0;
}
```

Output:

Enter numeric password: *****

Experiment No. 3 (a)

Caesar Cipher

Program in C++:

```
// A C++ program to illustrate Caesar Cipher Technique
#include <iostream>
using namespace std;

// This function receives text and shift and
// returns the encrypted text
string encrypt(string text, int s)
{
    string result = "";

    // traverse text
    for (int i=0;i<text.length();i++)
    {
        // apply transformation to each character
        // Encrypt Uppercase letters
        if (isupper(text[i]))
            result += char(int(text[i]+s-65)%26 +65);

        // Encrypt Lowercase letters
        else
            result += char(int(text[i]+s-97)%26 +97);
    }

    // Return the resulting string
    return result;
}

// Driver program to test the above function
int main()
{
    string text="ATTACKATONCE";
    int s = 4;
    cout << "Text : " << text;
    cout << "\nShift: " << s;
    cout << "\nCipher: " << encrypt(text, s);
    return 0;
}
```

Output:

Text : ATTACKATONCE
Shift: 4
Cipher: EXXEGOEXSRGI

Experiment No. 3 (b)

Substitution Technique

Program in Python:

```
# Python program to demonstrate  
# Substitution Cipher
```

```
import string
```

```
# A list containing all characters  
all_letters= string.ascii_letters
```

```
""""  
create a dictionary to store the substitution  
for the given alphabet in the plain text  
based on the key  
""""
```

```
dict1 = {}  
key = 4
```

```
for i in range(len(all_letters)):  
    dict1[all_letters[i]] = all_letters[(i+key)%len(all_letters)]
```

```
plain_txt= "I am studying Data Encryption"  
cipher_txt=[]
```

```
# loop to generate ciphertext
```

```
for char in plain_txt:  
    if char in all_letters:  
        temp = dict1[char]  
        cipher_txt.append(temp)  
    else:  
        temp =char  
        cipher_txt.append(temp)
```

```
cipher_txt= "".join(cipher_txt)  
print("Cipher Text is: ",cipher_txt)
```

```
""""  
create a dictionary to store the substitution  
for the given alphabet in the cipher  
text based on the key  
""""
```

```
dict2 = {}
for i in range(len(all_letters)):
    dict2[all_letters[i]] = all_letters[(i-key)%(len(all_letters))]

# loop to recover plain text
decrypt_txt = []

for char in cipher_txt:
    if char in all_letters:
        temp = dict2[char]
        decrypt_txt.append(temp)
    else:
        temp = char
        decrypt_txt.append(temp)

decrypt_txt = "".join(decrypt_txt)
print("Recovered plain text :", decrypt_txt)
```

Output:

Cipher Text is: M eq wxyhCmrk Hexe IrgvCtxmsr
Recovered plain text : I am studying Data Encryption

Experiment No. 5

Symmetric and Asymmetric Cryptography using RSA Algorithm

Program in C: Program of RSA Algorithm

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
#include<string.h>
long int p,q,n,t,flag,e[100],d[100],temp[100],j,m[100],en[100],i;
char msg[100];
int prime(long int);
void ce();
long int cd(long int);
void encrypt();
void decrypt();
int main()
{
    printf("\nENTER FIRST PRIME NUMBER\n");
    scanf("%ld",&p);
    flag=prime(p);
    if(flag==0)
    {
        printf("\nWRONG INPUT\n");
        exit(1);
    }
    printf("\nENTER ANOTHER PRIME NUMBER\n");
    scanf("%ld",&q);
    flag=prime(q);
    if(flag==0||p==q)
    {
        printf("\nWRONG INPUT\n");
        exit(1);
    }
    printf("\nENTER MESSAGE\n");
    fflush(stdin);
    scanf("%s",msg);
    for(i=0;msg[i]!=NULL;i++)
        m[i]=msg[i];
    n=p*q;
    t=(p-1)*(q-1);
    ce();
    printf("\nPOSSIBLE VALUES OF e AND d ARE\n");
    for(i=0;i<t-1;i++)
        printf("\n%ld\t%ld",e[i],d[i]);
    encrypt();
    decrypt();
    return 0;
}
int prime(long int pr)
{
    int i;
```



```

j=sqrt(pr);
for(i=2;i<=j;i++)
{
    if(pr%i==0)
        return 0;
}
return 1;
}
void ce()
{
    int k;
    k=0;
    for(i=2;i<t;i++)
    {
        if(t%i==0)
            continue;
        flag=prime(i);
        if(flag==1&&i!=p&&i!=q)
        {
            e[k]=i; flag=cd(e[k]);
            if(flag>0)
            {
                d[k]=flag;
                k++;
            }
            if(k==99)
                break;
        }
    }
}
long int cd(long int x)
{
    long int k=1;
    while(1)
    {
        k=k+t;
        if(k%x==0)
            return(k/x);
    }
}
void encrypt()
{
    long int pt,ct,key=e[0],k,len;
    i=0;
    len=strlen(msg);
    while(i!=len)
    {
        pt=m[i];
        pt=pt-96;
        k=1;
        for(j=0;j<key;j++)
        {
            k=k*pt;
            k=k%n;

```

```

    }
    temp[i]=k;
    ct=k+96;
    en[i]=ct;
    i++;
}
en[i]=-1;
printf("\nTHE ENCRYPTED MESSAGE IS\n");
for(i=0;en[i]!=-1;i++)
    printf("%c",en[i]);
}
void decrypt()
{
    long int pt,ct,key=d[0],k;
    i=0;
    while(en[i]!=-1)
    {
        ct=temp[i];
        k=1;
        for(j=0;j<key;j++)
        {
            k=k*ct;
            k=k%n;
        }
        pt=k+96;
        m[i]=pt;
        i++;
    }
    m[i]=-1;
    printf("\nTHE DECRYPTED MESSAGE IS\n");
    for(i=0;m[i]!=-1;i++)
        printf("%c",m[i]);
}

```

Output:

ENTER FIRST PRIME NUMBER

3

ENTER ANOTHER PRIME NUMBER

7

ENTER MESSAGE

hello

POSSIBLE VALUES OF e AND d ARE

5 5

11 11

THE ENCRYPTED MESSAGE IS

hqcco

THE DECRYPTED MESSAGE IS

hello

Experiment No. 6(a)

Implementation of DES Encryption

Program in C:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
void main()
{
    int i,ch,lp;
    char cipher[50],plain[50];
    char key[50];
    while(1)
    {
        printf("\n__Menu__\n");
        printf("\n 1:data encryption\b\n\n 2:data decryption\t\n\n 3:exit");
        printf("\n enter your choice=");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:printf("\n data encryption");
                    printf("\n enter the plain text");
                    fflush(stdin);
                    gets(plain);
                    printf("\n enter the encryption key:");
                    gets(key);
                    lp=strlen(key);
                    for(i=0;plain[i]!='\0';i++)
                        cipher[i]=plain[i]^lp;
                    cipher[i]='\0';
                    printf("\n the encrypted text is:");
                    puts(cipher);
                    break;
            case 2:
                    printf("\n decrypted text is:");
                    puts(plain);
                    break;
            case 3:exit(0);
        }
        getch();
    }
}
```

Output:

__Menu__

1:data encryption

2:data decryption

3:exit

enter your choice=1

data encryption

enter the plain texthello

enter the encryption key:2

the encrypted text is:ldmmn

__Menu__

1:data encryption

2:data decryption

3:exit

enter your choice=

Experiment No. 6(b)

Implementation of DES Decryption

Program in C++:

```
// Including dependencies
#include <iostream>
#include <string>
#include <cmath>
using namespace std;
// Array to hold 16 keys
string round_keys[16];
// String to hold the plain text
string pt;
// Function to convert a number in decimal to binary
string convertDecimalToBinary(int decimal)
{
    string binary;
    while(decimal != 0) {
        binary = (decimal % 2 == 0 ? "0" : "1") + binary;
        decimal = decimal/2;
    }
    while(binary.length() < 4){
        binary = "0" + binary;
    }
    return binary;
}
// Function to convert a number in binary to decimal
int convertBinaryToDecimal(string binary)
{
    int decimal = 0;
    int counter = 0;
    int size = binary.length();
    for(int i = size-1; i >= 0; i--)
    {
        if(binary[i] == '1'){
            decimal += pow(2, counter);
        }
        counter++;
    }
    return decimal;
}
// Function to do a circular left shift by 1
string shift_left_once(string key_chunk){
    string shifted="";
    for(int i = 1; i < 28; i++){
        shifted += key_chunk[i];
    }
    shifted += key_chunk[0];
    return shifted;
}
// Function to do a circular left shift by 2
string shift_left_twice(string key_chunk){
```

```

string shifted="";
for(int i = 0; i < 2; i++){
    for(int j = 1; j < 28; j++){
        shifted += key_chunk[j];
    }
    shifted += key_chunk[0];
    key_chunk= shifted;
    shifted ="";
}
return key_chunk;
}
// Function to compute xor between two strings
string Xor(string a, string b){
    string result = "";
    int size = b.size();
    for(int i = 0; i < size; i++){
        if(a[i] != b[i]){
            result += "1";
        }
        else{
            result += "0";
        }
    }
    return result;
}
// Function to generate the 16 keys.
void generate_keys(string key){
    // The PC1 table
    int pc1[56] = {
        57,49,41,33,25,17,9,
        1,58,50,42,34,26,18,
        10,2,59,51,43,35,27,
        19,11,3,60,52,44,36,
        63,55,47,39,31,23,15,
        7,62,54,46,38,30,22,
        14,6,61,53,45,37,29,
        21,13,5,28,20,12,4
    };
    // The PC2 table
    int pc2[48] = {
        14,17,11,24,1,5,
        3,28,15,6,21,10,
        23,19,12,4,26,8,
        16,7,27,20,13,2,
        41,52,31,37,47,55,
        30,40,51,45,33,48,
        44,49,39,56,34,53,
        46,42,50,36,29,32
    };
    // 1. Compressing the key using the PC1 table
    string perm_key = "";
    for(int i = 0; i < 56; i++){
        perm_key+= key[pc1[i]-1];
    }
}

```

```

// 2. Dividing the key into two equal halves
string left= perm_key.substr(0, 28);
string right= perm_key.substr(28, 28);
for(int i=0; i<16; i++){
    // 3.1. For rounds 1, 2, 9, 16 the key_chunks
    // are shifted by one.
    if(i == 0 || i == 1 || i==8 || i==15 ){
        left= shift_left_once(left);
        right= shift_left_once(right);
    }
    // 3.2. For other rounds, the key_chunks
    // are shifted by two
    else{
        left= shift_left_twice(left);
        right= shift_left_twice(right);
    }
    // Combining the two chunks
    string combined_key = left + right;
    string round_key = "";
    // Finally, using the PC2 table to transpose the key bits
    for(int i = 0; i < 48; i++){
        round_key += combined_key[pc2[i]-1];
    }
    round_keys[i] = round_key;
}
}
// Implementing the algorithm
string DES(){
    // The initial permutation table
    int initial_permutation[64] = {
        58,50,42,34,26,18,10,2,
        60,52,44,36,28,20,12,4,
        62,54,46,38,30,22,14,6,
        64,56,48,40,32,24,16,8,
        57,49,41,33,25,17,9,1,
        59,51,43,35,27,19,11,3,
        61,53,45,37,29,21,13,5,
        63,55,47,39,31,23,15,7
    };
    // The expansion table
    int expansion_table[48] = {
        32,1,2,3,4,5,4,5,
        6,7,8,9,8,9,10,11,
        12,13,12,13,14,15,16,17,
        16,17,18,19,20,21,20,21,
        22,23,24,25,24,25,26,27,
        28,29,28,29,30,31,32,1
    };
    // The substitution boxes. The should contain values
    // from 0 to 15 in any order.
    int substitution_boxes[8][4][16]=
    {{
        14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7,
        0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8,

```

```

4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0,
15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13
},
{
15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10,
3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5,
0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15,
13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9
},
{
10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8,
13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1,
13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7,
1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12
},
{
7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15,
13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9,
10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4,
3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14
},
{
2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9,
14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6,
4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14,
11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3
},
{
12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11,
10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8,
9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6,
4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13
},
{
4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1,
13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6,
1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2,
6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12
},
{
13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7,
1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2,
7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8,
2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11
}};

// The permutation table
int permutation_tab[32] = {
16,7,20,21,29,12,28,17,
1,15,23,26,5,18,31,10,
2,8,24,14,32,27,3,9,
19,13,30,6,22,11,4,25
};

// The inverse permutation table
int inverse_permutation[64]= {

```



```

40,8,48,16,56,24,64,32,
39,7,47,15,55,23,63,31,
38,6,46,14,54,22,62,30,
37,5,45,13,53,21,61,29,
36,4,44,12,52,20,60,28,
35,3,43,11,51,19,59,27,
34,2,42,10,50,18,58,26,
33,1,41,9,49,17,57,25
};
//1. Applying the initial permutation
string perm = "";
for(int i = 0; i < 64; i++){
    perm += pt[initial_permutation[i]-1];
}
// 2. Dividing the result into two equal halves
string left = perm.substr(0, 32);
string right = perm.substr(32, 32);
// The plain text is encrypted 16 times
for(int i=0; i<16; i++) {
    string right_expanded = "";
    // 3.1. The right half of the plain text is expanded
    for(int i = 0; i < 48; i++) {
        right_expanded += right[expansion_table[i]-1];
    }; // 3.3. The result is xored with a key
    string xored = Xor(round_keys[i], right_expanded);
    string res = "";
    // 3.4. The result is divided into 8 equal parts and passed
    // through 8 substitution boxes. After passing through a
    // substitution box, each box is reduces from 6 to 4 bits.
    for(int i=0;i<8; i++){
        // Finding row and column indices to lookup the
        // substitution box
        string row1= xored.substr(i*6,1) + xored.substr(i*6 + 5,1);
        int row = convertBinaryToDecimal(row1);
        string col1 = xored.substr(i*6 + 1,1) + xored.substr(i*6 + 2,1) +
xored.substr(i*6 + 3,1) + xored.substr(i*6 + 4,1);;
        int col = convertBinaryToDecimal(col1);
        int val = substitution_boxes[i][row][col];
        res += convertDecimalToBinary(val);
    }
    // 3.5. Another permutation is applied
    string perm2 = "";
    for(int i = 0; i < 32; i++){
        perm2 += res[permutation_tab[i]-1];
    }
    // 3.6. The result is xored with the left half
    xored = Xor(perm2, left);
    // 3.7. The left and the right parts of the plain text are swapped
    left = xored;
    if(i < 15){
        string temp = right;
        right = xored;
        left = temp;
    }
}

```

```

    }
    // 4. The halves of the plain text are applied
    string combined_text = left + right;
    string ciphertext = "";
    // The inverse of the initial permuttaion is applied
    for(int i = 0; i < 64; i++){
        ciphertext+= combined_text[inverse_permutation[i]-1];
    }
    //And we finally get the cipher text
    return ciphertext;
}
int main(){
    // A 64 bit key
    string key=
"1010101010111011000010010001100000100111001101101100110011011101";
    // A block of plain text of 64 bits
    pt=
"1010101111001101111001101010101111001101000100110010010100110110";
    string apt = pt;
    // Calling the function to generate 16 keys
    generate_keys(key);
    cout<<"Plain text: "<<pt<<endl;
    // Applying the algo
    string ct= DES();
    cout<<"Ciphertext: "<<ct<<endl;
    // Reversing the round_keys array for decryption
    int i = 15;
    int j = 0;
    while(i > j)
    {
        string temp = round_keys[i];
        round_keys[i] = round_keys[j];
        round_keys[j] = temp;
        i--;
        j++;
    }
    pt = ct;
    string decrypted = DES();
    cout<<"Decrypted text:"<<decrypted<<endl;
    // Comapring the initial plain text with the decrypted text
    if (decrypted == apt){
        cout<<"Plain text encrypted and decrypted successfully."<<endl;
    }
}

```

Output:

Plain text:

1010101111001101111001101010101111001101000100110010010100110110

Ciphertext:

1001111000100110100111110101101011111010010011011011101101110000

Decrypted

text:1010101111001101111001101010101111001101000100110010010100110110

Plain text encrypted and decrypted successfully.

Experiment No. 6(c)

Generating Keys for DES Decryption

Program in C++:

```
// Including dependancies
#include <iostream>
#include <string>
using namespace std;
// Array to hold the 16 keys
string round_keys[16];
// Function to do a circular left shift by 1
string shift_left_once(string key_chunk){
    string shifted="";
    for(int i = 1; i < 28; i++){
        shifted += key_chunk[i];
    }
    shifted += key_chunk[0];
    return shifted;
}
// Function to do a circular left shift by 2
string shift_left_twice(string key_chunk){
    string shifted="";
    for(int i = 0; i < 2; i++){
        for(int j = 1; j < 28; j++){
            shifted += key_chunk[j];
        }
        shifted += key_chunk[0];
        key_chunk= shifted;
        shifted ="";
    }
    return key_chunk;
}
void generate_keys(string key){
    // The PC1 table
    int pc1[56] = {
        57,49,41,33,25,17,9,
        1,58,50,42,34,26,18,
        10,2,59,51,43,35,27,
        19,11,3,60,52,44,36,
        63,55,47,39,31,23,15,
        7,62,54,46,38,30,22,
        14,6,61,53,45,37,29,
        21,13,5,28,20,12,4
    };
    // The PC2 table
    int pc2[48] = {
        14,17,11,24,1,5,
        3,28,15,6,21,10,
        23,19,12,4,26,8,
        16,7,27,20,13,2,
        41,52,31,37,47,55,
```

```

30,40,51,45,33,48,
44,49,39,56,34,53,
46,42,50,36,29,32
};
// 1. Compressing the key using the PC1 table
string perm_key = "";
for(int i = 0; i < 56; i++){
    perm_key+= key[pc1[i]-1];
}
// 2. Dividing the result into two equal halves
string left= perm_key.substr(0, 28);
string right= perm_key.substr(28, 28);
// Generating 16 keys
for(int i=0; i<16; i++){
    // 3.1. For rounds 1, 2, 9, 16 the key_chunks
    // are shifted by one.
    if(i == 0 || i == 1 || i==8 || i==15 ){
        left= shift_left_once(left);
        right= shift_left_once(right);
    }
    // 3.2. For other rounds, the key_chunks
    // are shifted by two
    else{
        left= shift_left_twice(left);
        right= shift_left_twice(right);
    }
// 4. The chunks are combined
string combined_key = left + right;
string round_key = "";
// 5. Finally, the PC2 table is used to transpose
// the key bits
for(int i = 0; i < 48; i++){
    round_key += combined_key[pc2[i]-1];
}
round_keys[i] = round_key;
cout<<"Key "<<i+1<<": "<<round_keys[i]<<endl;
}

}

int main(){
    string key = "10101010101110110000100100011000001001110011"
    "01101100110011011101";
    generate_keys(key);
}

```

Output:

```

Key 1: 000110010100110011010000011100101101111010001100
Key 2: 010001010110100001011000000110101011110011001110
Key 3: 000001101110110110100100101011001111010110110101
Key 4: 110110100010110100000011001010110110111011100011
Key 5: 011010011010011000101001111111101100100100010011
Key 6: 110000011001010010001110100001110100011101011110
Key 7: 011100001000101011010010110111011011001111000000

```

Key 8: 001101001111100000100010111100001100011001101101
Key 9: 1000010010111101101000100011100111101110011001100
Key 10: 000000100111011001010111000010001011010110111111
Key 11: 011011010101010101100000101011110111110010100101
Key 12: 110000101100000111101001011010100100101111110011
Key 13: 100110011100001100010011100101111100100100011111
Key 14: 001001010001101110001011110001110001011111010000
Key 15: 001100110011000011000101110110011010001101101101
Key 16: 000110000001110001011101011101011100011001101101

Experiment No. 7

Implementation of AES

Program in Python:

```
# -*- coding: utf-8 -*-
"""
Created on Sat Jan 29 16:32:25 2022

@author: PICT
"""

#!/usr/bin/python3

# Simplified AES implementation in Python 3
#
#=====

import sys

# ----- Decimal to 16 digit Binary -----

def decimalToBinary(n):
    return "{0:016b}".format(int(n))

# -----
# ----- Start of AES Algorithm -----

# S-Box
sBox = [0x9, 0x4, 0xa, 0xb, 0xd, 0x1, 0x8, 0x5,
        0x6, 0x2, 0x0, 0x3, 0xc, 0xe, 0xf, 0x7]

# Inverse S-Box
sBoxI = [0xa, 0x5, 0x9, 0xb, 0x1, 0x7, 0x8, 0xf,
         0x6, 0x0, 0x2, 0x3, 0xc, 0x4, 0xd, 0xe]

# Round keys: K0 = w0 + w1; K1 = w2 + w3; K2 = w4 + w5
w = [None] * 6

def mult(p1, p2):
    """Multiply two polynomials in GF(2^4)/x^4 + x + 1"""
    p = 0
    while p2:
        if p2 & 0b1:
            p ^= p1
        p1 <<= 1
        if p1 & 0b10000:
            p1 ^= 0b11
        p2 >>= 1
    return p & 0b1111
```

```

def intToVec(n):
    """Convert a 2-byte integer into a 4-element vector"""
    return [n >> 12, (n >> 4) & 0xf, (n >> 8) & 0xf, n & 0xf]

def vecToInt(m):
    """Convert a 4-element vector into 2-byte integer"""
    return (m[0] << 12) + (m[2] << 8) + (m[1] << 4) + m[3]

def addKey(s1, s2):
    """Add two keys in GF(2^4)"""
    return [i ^ j for i, j in zip(s1, s2)]

def sub4NibList(sbox, s):
    """Nibble substitution function"""
    return [sbox[e] for e in s]

def shiftRow(s):
    """ShiftRow function"""
    return [s[0], s[1], s[3], s[2]]

def keyExp(key):
    """Generate the three round keys"""
    def sub2Nib(b):
        """Swap each nibble and substitute it using sBox"""
        return sBox[b >> 4] + (sBox[b & 0xf] << 4)

    Rcon1, Rcon2 = 0b10000000, 0b00110000
    w[0] = (key & 0xff00) >> 8
    w[1] = key & 0x00ff
    w[2] = w[0] ^ Rcon1 ^ sub2Nib(w[1])
    w[3] = w[2] ^ w[1]
    w[4] = w[2] ^ Rcon2 ^ sub2Nib(w[3])
    w[5] = w[4] ^ w[3]

def encrypt(plaintext):
    """Encrypt plaintext block"""
    def mixCol(s):
        return [s[0] ^ mult(4, s[2]), s[1] ^ mult(4, s[3]),
                s[2] ^ mult(4, s[0]), s[3] ^ mult(4, s[1])]

    state = intToVec(((w[0] << 8) + w[1]) ^ plaintext)
    state = mixCol(shiftRow(sub4NibList(sBox, state)))
    state = addKey(intToVec((w[2] << 8) + w[3]), state)
    state = shiftRow(sub4NibList(sBox, state))
    return vecToInt(addKey(intToVec((w[4] << 8) + w[5]), state))

def decrypt(ctext):
    """Decrypt ciphertext block"""
    def iMixCol(s):
        return [mult(9, s[0]) ^ mult(2, s[2]), mult(9, s[1]) ^ mult(2, s[3]),
                mult(9, s[2]) ^ mult(2, s[0]), mult(9, s[3]) ^ mult(2, s[1])]

    state = intToVec(((w[4] << 8) + w[5]) ^ ctext)
    state = sub4NibList(sBoxI, shiftRow(state))

```

```

state = iMixCol(addKey(intToVec((w[2] << 8) + w[3]), state))
state = sub4NibList(sBoxI, shiftRow(state))
return vecToInt(addKey(intToVec((w[0] << 8) + w[1]), state))

# ----- End of AES Algorithm -----

if __name__ == '__main__':
    # Test vectors from "Simplified AES" (Steven Gordon)
    # (http://hw.siiit.net/files/001283.pdf)0010010011101100

    # Enter the Plain Text & Key -----

    print("\n-----")
    print("---AES - ADVANCE ENCRYPTION STANDARD ALGORIHTM ---")
    print("-----")

    pt = input("Enter a number as Plain Text (0 - 65000) : ")
    plaintext = format(int(pt), '#016b')
    print ("\nPlain Text : ", plaintext)

    key = input("Enter a number as Key (0 - 65000) : ")
    key = format(int(key), '#016b')
    print ("\nEncryption Key : ",key)

    keyExp(int(key, base=0))

    #----- Performing Encryption-----
    print("\n-----")
    print("----- Perfroming AES - ENCRYPTION ALGORIHTM ---")

    ct= encrypt(int(plaintext, base=0))
    ciphertext = format(int(ct), '#016b')
    print ("\nCipher Text in Binary : ",ciphertext)
    print ("\nCipher Text in Decimal : ",ct)

    print("\n-----\n")

    #----- Performing decryption-----

    print("\n-----")
    print("----- Perfroming AES - DECRYPTION ALGORIHTM ---")

    print ("\nCipher Text in Binary : ",ciphertext)
    print ("\nCipher Text in Decimal : ",ct)

    pt = decrypt(int(ciphertext, base=0))
    plaintext = format(int(pt), '#016b')
    print ("\nPlain Text in Binary : ",plaintext)
    print ("\nPlain Text in Decimal : ",pt)

    print("\n-----\n")
    print("----- End of AES Program -----")

```


sys.exit()

Output:

---AES - ADVANCE ENCRYPTION STANDARD ALGORIHTM ---

Enter a number as Plain Text (0 - 65000) : 5

Plain Text : 0b00000000000101

Enter a number as Key (0 - 65000) : 8

Encryption Key : 0b00000000001000

-----Perfroming AES - ENCRYPTION ALGORIHTM ---

Cipher Text in Binary : 0b1001011001100110

Cipher Text in Decimal : 38502

-----Perfroming AES - DECRYPTION ALGORIHTM ---

Cipher Text in Binary : 0b1001011001100110

Cipher Text in Decimal : 38502

Plain Text in Binary : 0b00000000000101

Plain Text in Decimal : 5

----- End of AES Program -----

Experiment No. 9

DH Algorithm

Program in C:

```
/* This program calculates the Key for two persons
using the Diffie-Hellman Key exchange algorithm */

#include<stdio.h>
#include<math.h>

// Power function to return value of  $a^b \bmod P$ 
long long int power(long long int a, long long int b,
                    long long int P)
{
    if (b == 1)
        return a;

    else
        return (((long long int)pow(a, b)) % P);
}

//Driver program
int main()
{
    long long int P, G, x, a, y, b, ka, kb;

    // Both the persons will be agreed upon the
    // public keys G and P
    P = 23; // A prime number P is taken
    printf("The value of P : %lld\n", P);

    G = 9; // A primitive root for P, G is taken
    printf("The value of G : %lld\n\n", G);

    // Alice will choose the private key a
    a = 4; // a is the chosen private key
    printf("The private key a for Alice : %lld\n", a);
    x = power(G, a, P); // gets the generated key

    // Bob will choose the private key b
    b = 3; // b is the chosen private key
    printf("The private key b for Bob : %lld\n\n", b);
    y = power(G, b, P); // gets the generated key

    // Generating the secret key after the exchange
    // of keys
    ka = power(y, a, P); // Secret key for Alice
    kb = power(x, b, P); // Secret key for Bob

    printf("Secret key for the Alice is : %lld\n", ka);
```

```
        printf("Secret Key for the Bob is : %lld\n", kb);  
  
        return 0;  
    }
```

Output:

The value of P : 23

The value of G : 9

The private key a for Alice : 4

The private key b for Bob : 3

Secret key for the Alice is : 9

Secret Key for the Bob is : 9